

LLM_Feedback

PROMPT:

Act as a product manager and senior CV engineer for [MY USE-CASE]. Are my targets realistic? Do my metrics and liveness design match the risk/UX? Critique data coverage, threshold selection, fairness/privacy, and on-device constraints. Propose concrete fixes. use case here:

EcoVault Face Unlock for Field Research Data I am designing a face unlock system for an app called EcoVault. Environmental researchers use this app on rugged laptops and phones to store sensitive data like endangered species locations, field notes, and water quality surveys. The goal is to use a standard RGB camera to verify the user right after they enter a PIN, so the app adds a second layer of protection without slowing people down too much.

Persona & Environment The main users are field biologists and ecologists working in places like wetlands, forests, park sites, and temporary field camps. They use rugged laptops and normal smartphones that may only have a standard RGB camera. The environment is inconsistent and hard to control. Sometimes they are in bright sunlight, sometimes in shade, and sometimes in dim tents or vehicles. They may also be wearing hats, sunglasses, masks, or rain gear. Because they often have no cell service, the full system has to run offline on the device.

Risk Level & Threats The risk level is moderate to high. The data stored in EcoVault can be sensitive enough that unauthorized access could expose endangered species locations or unpublished research. The main threats are realistic ones like a coworker, guide, or thief picking up an unattended device, knowing or guessing the PIN, and attempting to unlock the app with their own face or a simple photo of the enrolled user. I am focusing on these common unauthorized access attempts rather than advanced attacks like custom masks.

Privacy & Safety Constraints Privacy is a major part of this use case. All processing should happen locally on the device. No images or biometric data should be sent to a server. The system should store encrypted face embeddings instead of raw photos, present a clear consent screen during enrollment, and let the user fully delete their biometric profile and any logs at any time. The camera should only activate during an intentional unlock attempt and never run in the background.

Acceptance Targets The main metric for this project is TPR at a fixed FAR. I care more about that than plain accuracy because this is an authentication task with class imbalance, not a balanced classification problem. Since EcoVault uses face verification after a PIN, I am

treating it as a second-factor unlock. Under normal conditions, my target is TPR of at least 90% at FAR of at most 2%. Under harder conditions like darker lighting or partial face covering, my target is TPR of at least 80% at FAR of at most 5%. I also want EER of 8% or lower and AUC of at least 0.98.

For usability, I want the system to run at 100 ms or less on median inference, with 300 ms as a hard upper limit. That keeps the face check fast enough that it still feels worth using after a PIN. For deployment size, I want the model to stay under 20 MB if possible, with 25 MB as a hard cap, since field devices need space for offline maps, notes, photos, and survey data.”

UNEDITED FEEDBACK: “Your on-device constraints and privacy designs are solid, but your security thresholds are too lenient for the stated risk level, and relying on a standard RGB camera without presentation attack detection leaves the system vulnerable to photo attacks. Evaluating on a single celebrity identity from the LFW dataset completely fails to validate performance for diverse field researchers operating in extreme environments.

Metrics and Threshold Selection

Targeting a False Accept Rate (FAR) of 2% to 5% is too high for protecting sensitive data, as a 2% FAR gives an unauthorized user a 1-in-50 chance of bypassing the system. While face unlock serves as a second factor, it must act as a strong barrier if the primary PIN is compromised via shoulder surfing. Industry standards for mobile authentication require an FAR of 0.01% to 0.1%, but maintaining an 80-90% True Positive Rate (TPR) at these thresholds with occluded faces requires specialized architectures.

Liveness and Threat Mitigation

Your threat model explicitly includes photo attacks, but standard facial recognition embeddings generated in your baseline only verify identity without evaluating liveness. These models confidently match a high-quality printed photo or digital screen to the enrolled user. Because you are restricted to a standard RGB camera, you must implement an RGB-based Presentation Attack Detection (PAD) model to block replay attacks.

Data Coverage and Fairness

The current evaluation relies entirely on the LFW dataset and isolates a single identity as the positive class, which is statistically unviable for a production system. LFW lacks the extreme environmental conditions your persona demands, such as dim lighting in tents or occlusion from rain gear. You must implement stratified demographic testing across skin tones and lighting conditions to prevent disparate impact for diverse field researchers.

On-Device Constraints

Your constraints of under 100 ms median latency and under 25 MB model size are realistic, as demonstrated by your MobileNetV2 baseline operating at 70.2 ms and 16.43 MB. This deployment budget must encompass the entire computer vision pipeline, including face detection, verification, and liveness models. The current approach of fine-tuning with binary cross-entropy is flawed for open-set face recognition because it does not optimize the embedding space for maximal inter-class separation.

Architectural Fixes

To optimize the embedding space and improve TPR at lower FARs, replace the binary cross-entropy classification head with an angular margin penalty loss like ArcFace or CosFace. You must overhaul your training pipeline by pre-training on a massive dataset like MS1M-RetinaFace and fine-tuning on datasets containing natural occlusions like IJB-C.

Deployment Fixes

Convert your models to TFLite INT8 to reduce the memory footprint by 4x and meet your hardware constraints. This compression allows you to fit a face detector, an RGB anti-spoofing classifier trained on OULU-NPU, and an embedding model like MobileFaceNet under your 25 MB cap. Establish demographic fairness metrics using datasets like Racial Faces in the Wild to ensure consistent performance across all user groups.

Are you open to allocating part of the 25 MB storage budget to a dedicated anti-spoofing model, or do you expect the primary verification model to implicitly handle liveness detection?”

Reflection on feedback

The feedback was actually really helpful in showing me how vulnerable my original design was and what it takes to build a secure app for the real world. I incorporated a lot of the structural suggestions, like moving away from binary cross-entropy and using an angular margin loss like ArcFace. That makes a lot more sense for actually separating identities. I also realized my security thresholds were way too relaxed. A 2% False Accept Rate is too risky for sensitive data, so I am adjusting the targets to be much stricter. Finally, I am factoring in a dedicated Presentation Attack Detection model because a standard RGB camera is definitely vulnerable to someone just holding up a photo.

A few things I chose not to do are mostly related to the data and extreme optimization. I am not going to shift my dataset away from LFW. If I had the time, I would have taken the pictures myself in different lighting and environments to match actual field conditions. I did not have the time for that, so I had to work with celebrity photos. That is okay with me right now because it gives me a lot more volume, which really helps with the actual modeling. I also decided not to pre-train on massive datasets like MS1M because that would blow up the scope of the project and require compute I do not need to spend just to prove the concept. I also chose not to push the False Accept Rate all the way down to the 0.01% industry standard. Field biologists are working in dim tents, wearing hats, and covered in rain gear. If the system is that strict, it will reject the actual users too often and ruin the usability. I am also holding off on doing the full TFLite INT8 compression to jam three different models under the 25 MB cap. I want to keep things simple and get the core verification logic right before I worry about hyper-optimizing the file size. The feedback really helped me understand the gap between a basic classification problem and a secure, real world authentication system.